TASK 1

Encoding Formats

ASCII:

It is a process of turning letters and symbols into numerical values. Symbols and letters are converted into numbers according to the predefined tables as shown in figure 1. (Bagci, 2025)

ASCII table

Char	Dec	Oct	Hex	Char	Dec	Oct	Hex	Char	Dec	Oct	Hex	Char	Dec	Oct	Hex
(nul)	0	0000	0x00	(sp)	32	0040	0x20	@	64	0100	0x40	`	96	0140	0x60
(soh)	1	0001	0x01	!	33	0041	0x21	A	65	0101	0x41	a	97	0141	0x61
(stx)	2	0002	0x02	"	34	0042	0x22	B	66	0102	0x42	b	98	0142	0x62
(etx)	3	0003	0x03	#	35	0043	0x23	C	67	0103	0x43	c	99	0143	0x63
(eot)	4	0004	0x04	$	36	0044	0x24	D	68	0104	0x44	d	100	0144	0x64
(enq)	5	0005	0x05	%	37	0045	0x25	E	69	0105	0x45	e	101	0145	0x65
(ack)	6	0006	0x06	&	38	0046	0x26	F	70	0106	0x46	f	102	0146	0x66
(bel)	7	0007	0x07	'	39	0047	0x27	G	71	0107	0x47	g	103	0147	0x67
(bs)	8	0010	0x08	(	40	0050	0x28	H	72	0110	0x48	h	104	0150	0x68
(ht)	9	0011	0x09	)	41	0051	0x29	I	73	0111	0x49	i	105	0151	0x69
(nl)	10	0012	0x0a	*	42	0052	0x2a	J	74	0112	0x4a	j	106	0152	0x6a
(vt)	11	0013	0x0b	+	43	0053	0x2b	K	75	0113	0x4b	k	107	0153	0x6b
(np)	12	0014	0x0c	,	44	0054	0x2c	L	76	0114	0x4c	l	108	0154	0x6c
(cr)	13	0015	0x0d	-	45	0055	0x2d	M	77	0115	0x4d	m	109	0155	0x6d
(so)	14	0016	0x0e	.	46	0056	0x2e	N	78	0116	0x4e	n	110	0156	0x6e
(si)	15	0017	0x0f	/	47	0057	0x2f	O	79	0117	0x4f	o	111	0157	0x6f
(dle)	16	0020	0x10	0	48	0060	0x30	P	80	0120	0x50	p	112	0160	0x70
(dc1)	17	0021	0x11	1	49	0061	0x31	Q	81	0121	0x51	q	113	0161	0x71
(dc2)	18	0022	0x12	2	50	0062	0x32	R	82	0122	0x52	r	114	0162	0x72
(dc3)	19	0023	0x13	3	51	0063	0x33	S	83	0123	0x53	s	115	0163	0x73
(dc4)	20	0024	0x14	4	52	0064	0x34	T	84	0124	0x54	t	116	0164	0x74
(nak)	21	0025	0x15	5	53	0065	0x35	U	85	0125	0x55	u	117	0165	0x75
(syn)	22	0026	0x16	6	54	0066	0x36	V	86	0126	0x56	v	118	0166	0x76
(etb)	23	0027	0x17	7	55	0067	0x37	W	87	0127	0x57	w	119	0167	0x77
(can)	24	0030	0x18	8	56	0070	0x38	X	88	0130	0x58	x	120	0170	0x78
(em)	25	0031	0x19	9	57	0071	0x39	Y	89	0131	0x59	y	121	0171	0x79
(sub)	26	0032	0x1a	:	58	0072	0x3a	Z	90	0132	0x5a	z	122	0172	0x7a
(esc)	27	0033	0x1b	;	59	0073	0x3b	[	91	0133	0x5b	{	123	0173	0x7b
(fs)	28	0034	0x1c	<	60	0074	0x3c	\	92	0134	0x5c	|	124	0174	0x7c
(gs)	29	0035	0x1d	=	61	0075	0x3d	]	93	0135	0x5d	}	125	0175	0x7d
(rs)	30	0036	0x1e	>	62	0076	0x3e	^	94	0136	0x5e	~	126	0176	0x7e
(us)	31	0037	0x1f	?	63	0077	0x3f	_	95	0137	0x5f	(del)	127	0177	0x7f

Figure 1: ASCII Table (src : blogspot)

In command line, we can obtain ASCII value by using the command: echo -n “<Word>” | od –An –tu1. Here, echo will print the word in double quote, -n is used for new line, | (pipe) is to send output of LHS to RHS command, od is used to display the raw byte values, -An (-A and –n together) tells not to display address offsets, -tu1 is used for each character’s ASCII value in unsigned decimal form.

```
root@Pra-250546:/foundation# echo -n "A B C a b c" | od -An -tu1
  65  32  66  32  67  32  97  32  98  32  99
root@Pra-250546:/foundation# echo -n "ABCabc" | od -An -tu1
  65  66  67  97  98  99
```

Figure 3: Word to ASCII

We can also reverse this ASCII using command echo "<ASCII Values >" | awk '{for(i=1;i<=NF;i++) printf "%c",$i; print ""}'. Here, echo with print the word in double quote, | (pipe) is to send the output of left to right, awk is tool used to scan, split and process input data field by field, NF stores how many ASCII values are presents, for(i=1;i<=NF;i++) this loop iterates through each ASCII value one by one, printf "%c",$i used to convert the ASCII number into its corresponding character and print”” will print new line at the end of the output.

```
root@Pra-250546:/foundation# echo "65 32 66 32 67 32 97 32 98 32 99" | awk '
{for(i=1;i<=NF;i++) printf "%c",$i; print ""}'
A B C a b c
root@Pra-250546:/foundation# echo "65 66 67 97 98 99" | awk '{for(i=1;i<=NF;
i++) printf "%c",$i; print ""}'
ABCabc
```

Figure 3: ASCII to readable word

Strengths and Weakness of ASCII: (Bagci, 2025)

Strengths	Weakness
It is simple and universally supported	Character used here is limited
It is ideal for text-based protocols like SMTP	It cannot represent binary data directly
It has low processing overhead	It is inadequate for multimedia data

BASE64

Base 64 is an encoding scheme that breaks binary data into 6-bit segments, which are then checked with the predefined 64 character and corresponding readable letters are used. It contains 10 numeric values, 26 uppercase alphabets, 26 lowercase alphabets and 2 special characters. The table below is the output letter according to the 6-bit binary segments. (Josefsson, 2003)

Nr	Character		Nr	Character		Nr	Character		Nr	Character
0	A		16	Q		32	g		48	w
1	B		17	R		33	h		49	x
2	C		18	S		34	i		50	y
3	D		19	T		35	j		51	z
4	E		20	U		36	k		52	0
5	F		21	V		37	l		53	1
6	G		22	W		38	m		54	2
7	H		23	X		39	n		55	3
8	I		24	Y		40	o		56	4
9	J		25	Z		41	p		57	5
10	K		26	a		42	q		58	6
11	L		27	b		43	r		59	7
12	M		28	c		44	s		60	8
13	N		29	d		45	t		61	9
14	O		30	e		46	u		62	+
15	P		31	f		47	v		63	/

Figure 4: Base 64 table (src : onetechtips)

Words can be easily converted into Base64. The figure below shows the way to encode and decode in Base64.

```
root@Pra-250546:/foundation# echo "Hi Batch39C" | base64
SGkgQmF0Y2gzOUMK
root@Pra-250546:/foundation# echo "SGkgQmF0Y2gzOUMK" | base64 --decode
Hi Batch39C
root@Pra-250546:/foundation#
```

Figure 5: Encoded and Decoded text in Base64

Base64 is not an encryption method, it makes transmission of text safe over text-only protocols like email and web service.

Strengths and Weaknesses of Base64: (OnlineTinker, 2025)

Strengths:

- It enables binary data transmission over text-only protocols
- It is widely supported in HTTP and OAuth
- It preserves data integrity across the system

Weakness:

- It increases the data size by 33%. It can be seen in Figure 5

- It is often taken as encryption, leading to security misconfiguration
- It can be abused for payload obfuscation

HEX:

In Hex encoding 16 distinct symbols are used to represent numbers. 0-9 represents zero to nine and A-F represents ten to fifteen. It is used to represent binary data in a human readable format. (*Hexdecoder* 2024)

Decimal	Binary	Hexadecimal
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Figure 6: HEX Table (src: vectorstock)

In terminal we can convert any word into hexadecimal by using xxd and reverse it by using flag –r in xxd. The figure below shows the conversion of the word and reversing it.

```
root@Pra-250546:/foundation# echo -n "This is fun" | xxd -p
546869732069732066756e
root@Pra-250546:/foundation# echo "546869732069732066756e" | xxd -r -p; echo
This is fun
```

Figure 7: Encoded and decoded in Hex

Strengths and weakness of Hex: (*Hexdecoder* 2024)

Strengths:

- It represents binary data into human readble
- It is useful in debugging hashes and logs
- It is deterministic and simple to implement

Weakness:

- It doubles the data size
- It is inefficient for large payloads
- It is rarely used directly in web payload transmission

URL Encoding:

URL encoding is the way to represent characters safely in URLs. URLs only include certain characters, most of the characters from the ASCII set. The characters other than that are treated as unsafe or reserved. Character like <, >, “ , #, %, |, } ,] and others may break url when transmitted so before transmitting them it should be encoded. Reserved characters include ;, /, ?, @, = and & have special purpose in the URL and should be encoded if used literally. (Berners-Lee et al., 1994)

Character	ASCII Value	Encoded Value
`,`	`44` (0x2C)	`%2C`
`/`	`47` (0x2F)	`%2F`
`:`	`58` (0x3A)	`%3A`
`;`	`59` (0x3B)	`%3B`
`=`	`61` (0x3D)	`%3D`
`?`	`63` (0x3F)	`%3F`
`@`	`64` (0x40)	`%40`
`[`	`91` (0x5B)	`%5B`
`]`	`93` (0x5D)	`%5D`

Figure 4: URL Encoding (src : mubbiesnotebook)

Example of URL encoding

Unsafe URL → http://batch39.com/page#1 foundation

In this example URL there is # character which is a reserved symbol so it must be encoded as %23 and there is also space before the word foundation it should also be encoded as %20.

Safe URL → http://batch39.com/page%231%20foundation

Strengths and weakness of URL encoding. (Berners-Lee et al., 1994)

Strengths:

- It prevents URL parsing errors
- It is essential for query strigs and form submissions
- It helps mitigate injection vulnerabilities

Weakness:

- It increases the payload length
- Its misuse can lead to double-encoding vulnerabilities

Real-World Applications of Encoding

Showing the use of Base64 in email:

Email and web protocol prefer text-based transmission only. We cannot send file like image, pdf and others directly on the email they must convert into text based to send to others. This is not visible to email directly but behind every file is converted into Base64 before sending to others. We can check that by using email header analysis.

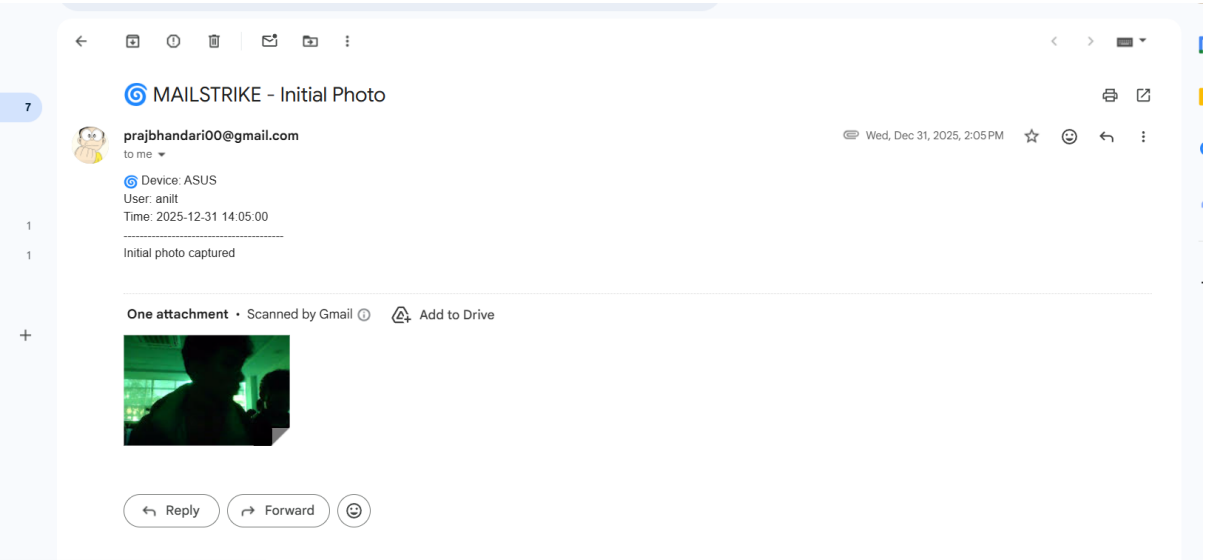

Figure 5: Email front

This is what users see when an image is sent via email that looks direct image but in backend, it is different we can see it clicking 3dots then show origin.

Download Original

Copy to clipboard


```
Return-Path: <prajbhandari00@gmail.com>
Received: from ASUS.domain.com ([110.44.114.6])
        by smtp.gmail.com with ESMTPSA id d9443c01a7336-2a2f3d6ec6bsm314823995ad.87.2025.12.31.00.20.10
        for <prajbhandari00@gmail.com>
        (version=TLS1_3 cipher=TLS_AES_256_GCM_SHA384 bits=256/256);
        Wed, 31 Dec 2025 00:20:11 -0800 (PST)
Message-ID: <6954dcbb.170a0220.34f07f.690b@mx.google.com>
Date: Wed, 31 Dec 2025 00:20:11 -0800 (PST)
Content-Type: multipart/mixed; boundary="===============0854576565771512709==="
MIME-Version: 1.0
From: prajbhandari00@gmail.com
To: prajbhandari00@gmail.com
Subject: 🌀 MAILSTRIKE - Initial Photo

--===============0854576565771512709==
Content-Type: text/plain; charset="utf-8"
MIME-Version: 1.0
Content-Transfer-Encoding: base64

8J+MgCBEZXZpY2U6IEFTVVMKVXNlcjogYW5pbHQKVGltZTogMjAyNS0xMi0zMSAxNDowNTowMAot
LS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tLS0tCkluaXRpYWwgcGhvdG8gY2Fw
dHVyZWQK
--===============0854576565771512709==
Content-Type: image/jpeg
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="initial_photo.jpg"


--===============0854576565771512709==--
```


Figure 6: Email origin

Image Base64 is not directly there but we can get by downloading the origin file and we get the image attached to it, in the Base64. SMTP safely transmits it as a text. Without Base64 the file would be corrupted.

```
--================0854576565771512709==
Content-Type: image/jpeg
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="initial_photo.jpg"

/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAIBAQEBAQIBAQECAgICAgQDAgICAgUEBAMEBgUGBgYF
BgYGBwkIBgcJBwYGCAsICQoKCgoKBggLDAsKDAkKCgr/2wBDAQICAgICAgUDAwUKBwYHCgoKCgoK
CgoKCgoKCgoKCgoKCgoKCgoKCgoKCgoKCgoKCgoKCgoKCgoKCgoKCgoKCgoKCgoKCgoKCgoKCgr/wAARCALQBQADASIA
AhEBAxEB/8QAHwAAAQUBAQEBAQEAAAAAAAAAAAECAwQFBgcICQoL/8QAtRAAAgEDAwIEAwUFBAQA
AAF9AQIDAAQRBRIhMUEGE1FhByJxFDKBkaEII0KxwRVS0fAkM2JyggkKFhcYGRolJicoKSo0NTY3
ODk6Q0RFRkdISUpTVFVWV1hZWmNkZWZnaGlqc3R1dnd4eXqDhIWGh4iJipKTlJWWl5iZmqKjpKWm
p6ipqrKztLW2t7i5usLDxMXGx8jJytLT1NXW19jZ2uHi4+Tl5ufo6erx8vP09fb3+Pn6/8QAHwEA
AwEBAQEBAQEBAQAAAAAAAAECAwQFBgcICQoL/8QAtREAAgECBAQDBAcFBAQAAQJ3AAECAxEEBSEx
BhJBUQdhcRMiMoEIFEKRobHBCSMzUvAVYnLRChYkNOEl8RcYGRomJygpKjU2Nzg5OkNERUZHSElK
U1RVVldYWVpjZGVmZ2hpanN0dXZ3eHl6goOEhYaHiImKkpOUlZaXmJmaoqOkpaanqKmqsrO0tba3
uLm6wsPExcbHyMnK0tPU1dbX2Nna4uPk5ebn6Onq8vP09fb3+Pn6/9oADAMBAAIRAxEAPwD8uJZf
tEWAaCPNiwbql+xG2j/0ekM+Tnj8q+Mo0p+z5T8EjVnKpzLYPsgpB9amlm/5dKimhro5KKLnDnJY
ZqjeUHr6VDNCYqIPtXnf9MqcJcxEqV+pbjdliqHzv31S+d++606p9rUq9UKjKUnqwpv/AC0ohzD5
X/PWifv+NS6X8ruHsfMlm+yw81FP3/Gj/lnUsMNrSVSovi0NKLrEUP7mGib99DU1Qwy+fVGblOXx
Mims+PtVEP7mGKr4+yzQ/ZSaozQ3Pnf9Mqyp+98WhpCvOBJD++m+1UT9/wAaIO34UeT7fpWvxU+W
Wo6lacCrP2q/psPnCiaEf8+tR2cjQS5xVVJci5ehDrxl8RsGHzulZeow+RNyK2dN/fdKNS0fzq44
e5XODmtU5paGXZw/2lDmuJ8R6bqeg66bm2/1c1x+/rtoYDZzZH+qo8R+G7XUrP8A0X97XfTqRw1T
mjFWPTweL+q1L390xtH1I3kMVbME3k2lcb4bvP7N1KXS7r97/wA+9dlZzfuawrR9hO0TtxtP2M+Z
u8fIJv3tc54jtDMeDXT+SPaqOsaaLyGiEuY46OI/ecqR5hfQGA5rP/1NdJrummKasCaz/fdK9Gjs
fXYTEQlT5ZFrR5v+nquj8n7ZZ1yVn+56V1GjzfuaitEzxuvw6mPq1rtrFmm8iut1iyPk8CuXu7cw
9quhVSKwVaLIoZv31dRo94fJ6VyMPT8K39HmrGsa4qnCauS69aedXLzReVN9a7fUrPzocfzrk9Xt
MTdfrxWmDqT9nyk4Kp0IIf8AX/hXSeG7zya5ez/czYrZ0eb99VYl851Yn36fKdHqUP2yHrXG6jDd
RTYNdvZ/vtNzXOa7ZZmrDDS5GedhcROFTlkYtbWgzeTWO1uVnq9ps3k812Vtj0KsOenzROyhl8+z
5rD1SImbGK19Cm86Gq+uWphPNebGXIfOU58mIaMTyfb9Kls5vJqtc9fwqSDt+FdSd1c9mMeaHMdl
4Vmuh/x63XlV29p/qq8+8LTXXnRV6Dpv/HpXDiY8h8tm/PB7ktX7P7LDDFdf+QaqeSPatDTIfJrn
fv8AxxHh16nMve1OA+NWntpU2j6nbW8PI8rzvXzq0vCE3nWcXH/LvUvxqiOr+Fv8AST/qLisv4b3n
2zR4rUD97BXscsP7Jjpse/F+0ySD6R6Fj4vWofwHI9ta4lF/DFcflXafBrUv7S02L/r3rH8WadR
8JanbLa+ZFBB9qqb9nu/tbjRbef7N5UspIqYS9pl3LE8yqoVsilG1+X+v0PdtI/fQ/6XUuu2h1HT
rq2uf9V/Z89RaPD/AKHFV+aG0ms7r7V+9i+z/wCprz6FTWx+cqrD6weBTHMUZ9ai/wCW34WCW1nW7
aG6+z/6oCDzKy5fFNtEftP2mGu2tSkfdUMLU9nzQNzzh7VHef6msuDXzLFkHzYvWibXu1KnGdP4k
SsJWVTlkrGbrkvlT59KqHWba0td1yf3XtVbW9TwOK4Hx34w8mL7La3VdlCn7Z7n02W5dPGyUbFf4
```


This is image in Base 64

Ln 30, Col 77 | 142,846 characters | Plain text

Figure 7: Image Base64

We can decode this and get the image. Several online tools are available for decoding this, but for this, I have created my own app to do this. App is available in GitHub. This demonstrates a weakness of Base64, it does need the cryptographic key to decode the data. Gmail use TLS-Based SMTP Email Transmission, so it is secure, but when we send data without encryption then it can easy capture by man-in-the-middle attacker and see the data inside it.

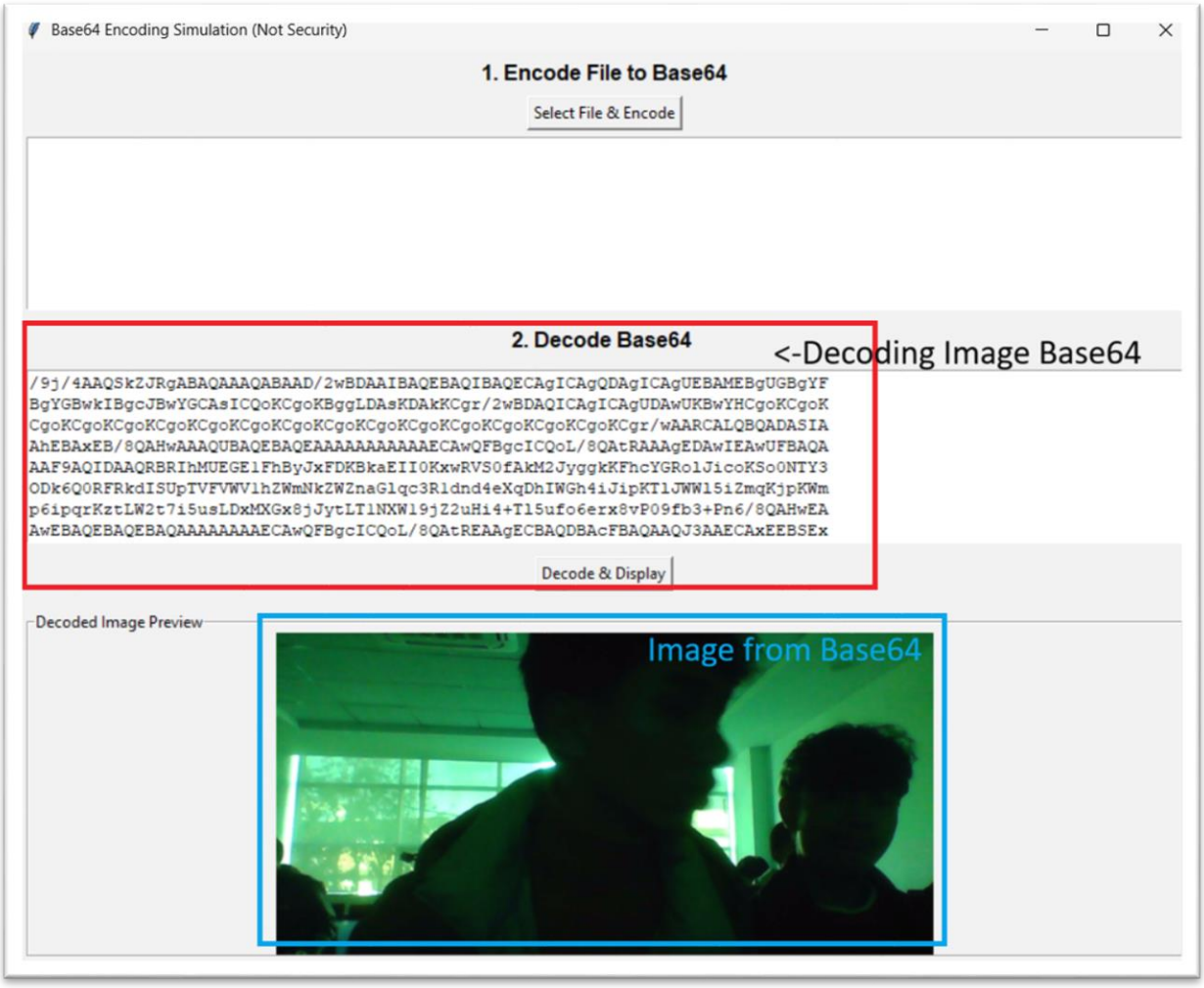

Figure 8: Image from Base64

This is the use of Base64. Secure application applies encryption methods before transmitting data. For simplicity below is the full process of transmission of file in email.

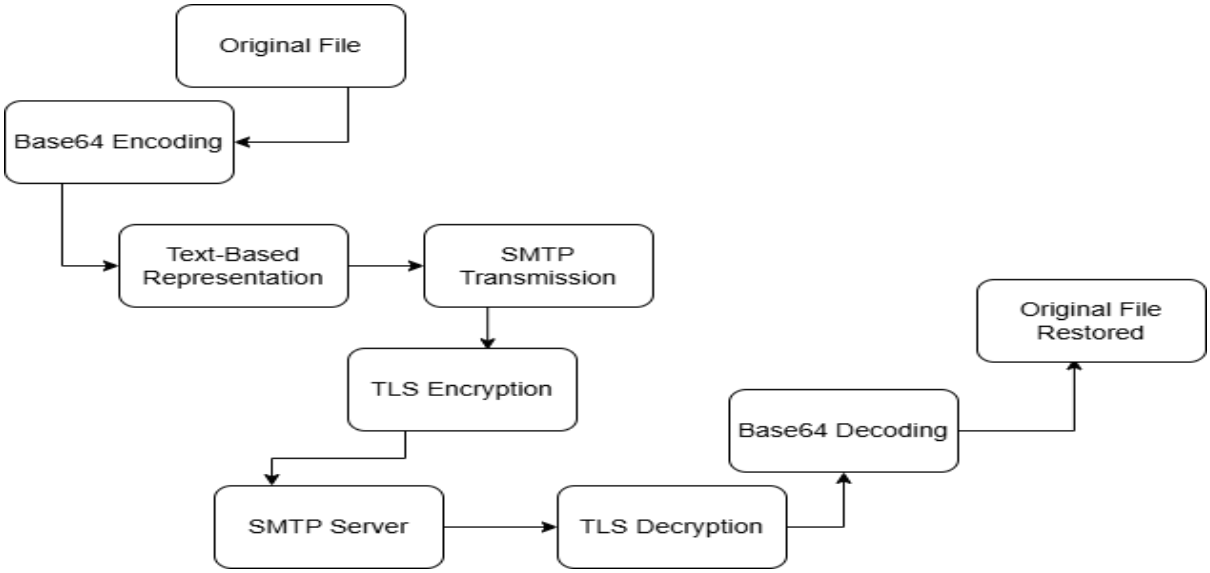

Figure 9: Base64 Encoding in TLS-Based SMTP Email Transmission

URL Encoding and Injection Attack Prevention

We have already covered the Safe character and dangerous character in URL. To show the danger I have setup a web that is vulnerable to the injection. Website code is available in the GitHub.

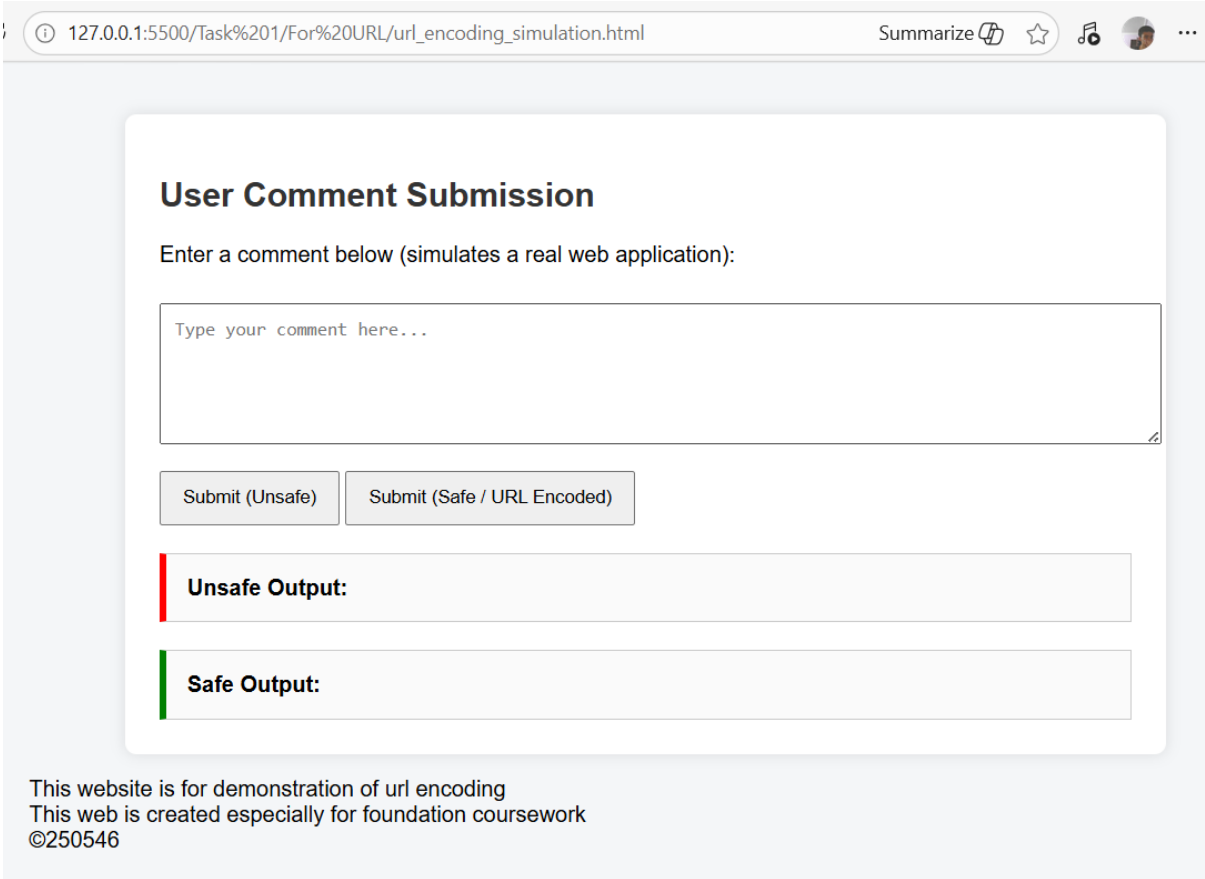

Figure 10: Vulnerable web

This web show, the role of URL encoding in preventing injection attacks. There are two submit button one shows an unsafe method (without URL encoding) and a safe method (with URL encoded). When we try to run script, it executes due to absence of URL encoding and do whatever the script says but when encoding is applied then it will show the script as a text, it does not execute.

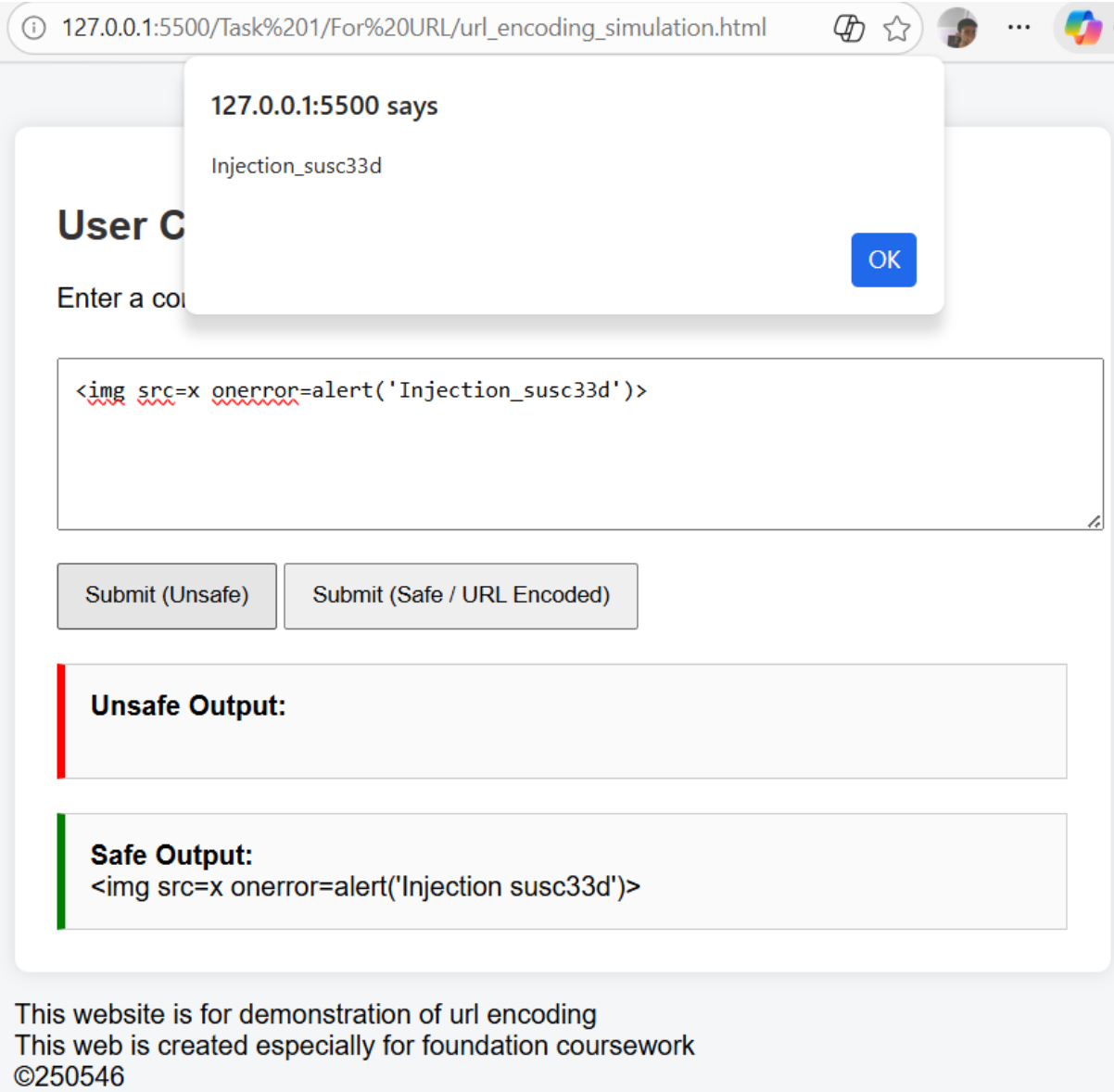

Figure 11: URL encoding

This illustrates that URL encoding prevents script execution and display the input script as a plain text, but when script is submitted through unsafe method the alert appeared which show success of injection attack on the web. If injection attack is not prevented, then database data can also be extracted.

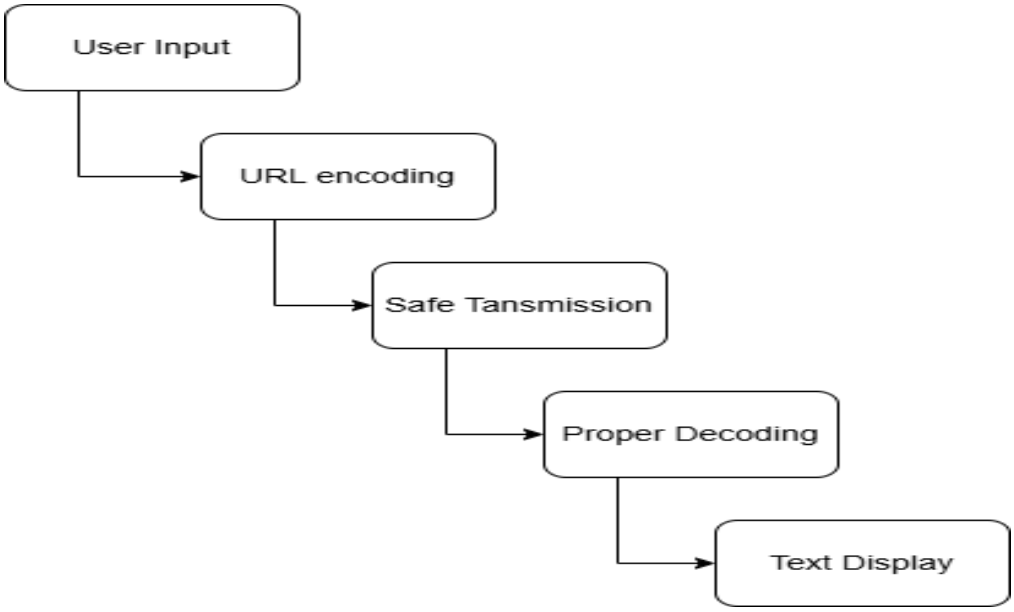

Figure 10: Data flow diagram of URL encoding

JWT Encoding in OAuth Authentication

JWT (Json Web Token) are widely used in modern web application and OAuth-based authentication to enable stateless user authentication. It used Base64URL encoding to safely transmit authentication data across HTTP/HTTPS without breaking rules. (Ibrahim, 2024) To simulate this a simple e-commerce web application is created and is available in GitHub with more details.

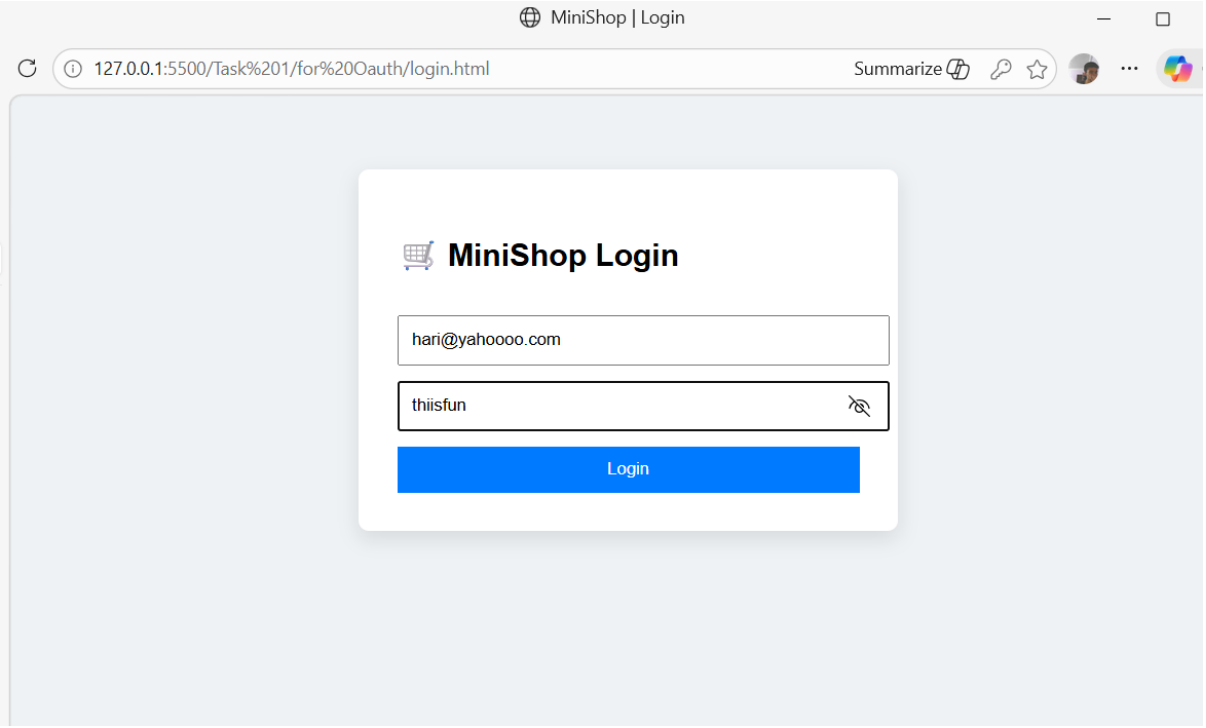

Figure 11: Login page to simulate OAuth

After we login, the application generates a JWT that contains user information such as email and role. The token payload is stored in browser's local storage and is not displayed to the user like real application do.

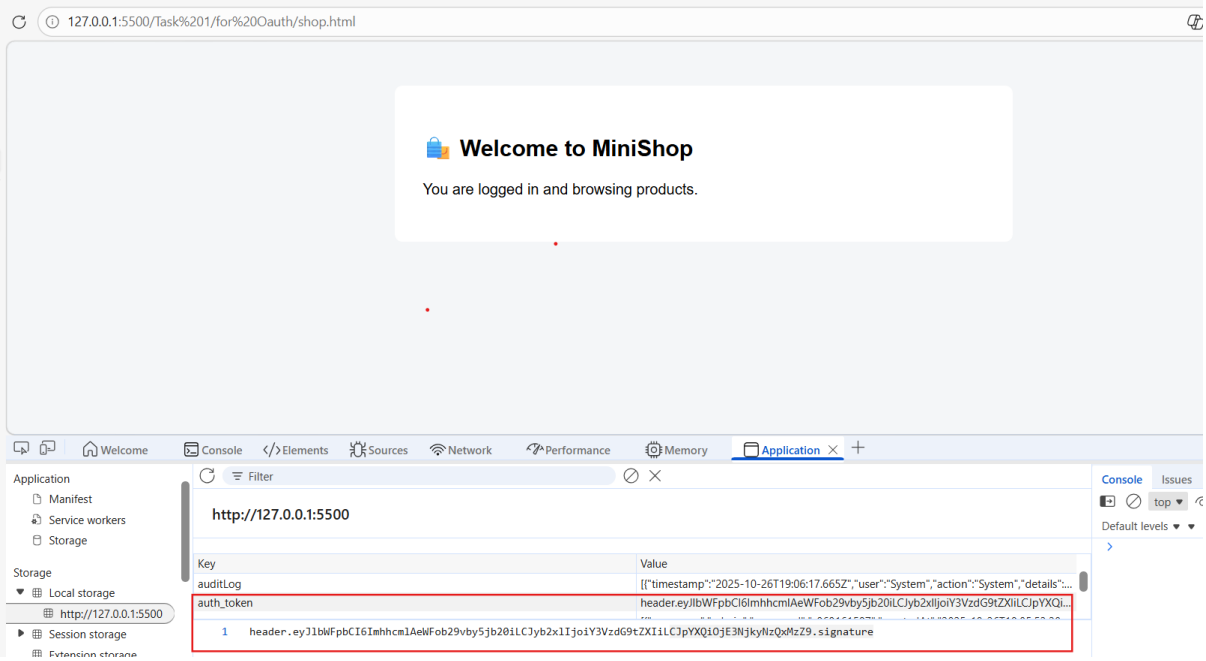

Figure 12: JWT token

To evaluate security risks, an attacker scenario was simulated by extracting the stored JWT from the browser and decoding its payload using Base64 decoding technique. The decoded tokens reveal user information without requiring any cryptographic key. This highlights the importance of encryption.

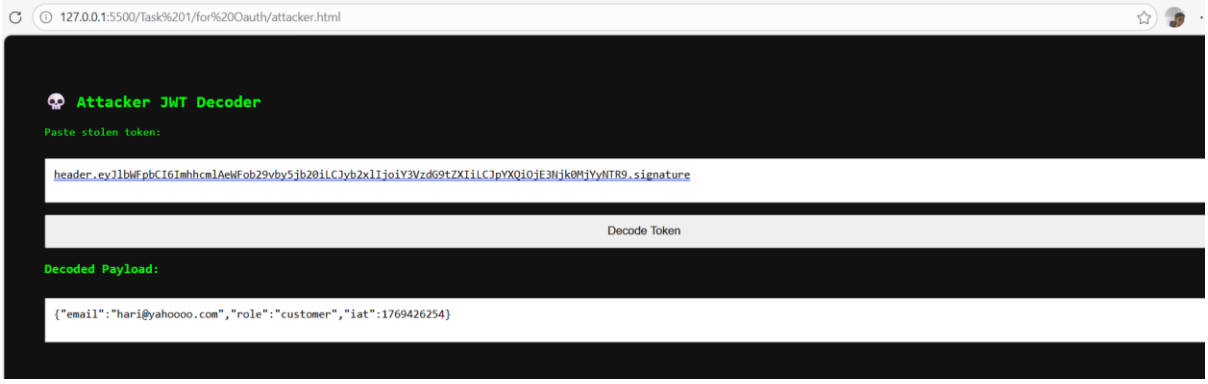

Figure 13: JWT Token decoding

Our simulated application closely mirrors real web authentication systems by using JWT – based login, client-side token storage and Base64 encoding. However, unlike production application our simulation lacks server-side verification and cryptographic signatures. This shows only Base64 cannot protect our data. Figure below representation of the process that we follow and mitigating measure.

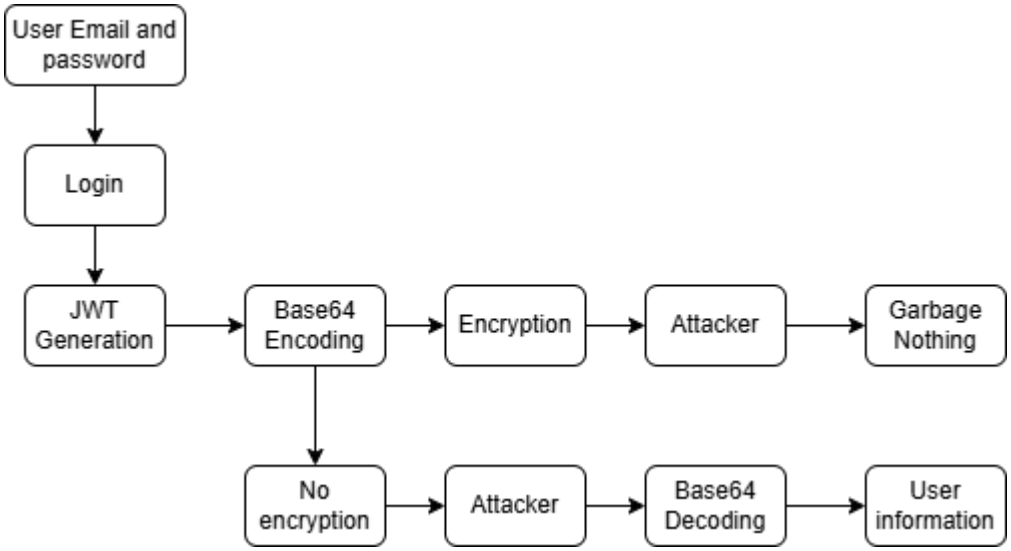

Figure 14: JWT Encoding Abuse Scenario

Base64-Encoded File Transfer over HTTP and HTTPS.

Modern web application transmits binary data such as images using text-based protocols. To ensures compatibility, such data is commonly encoded using base64 before transmission. To demonstrate this a simple web application was developed using browser-based client and python flask server.

Image transmission through HTTP only

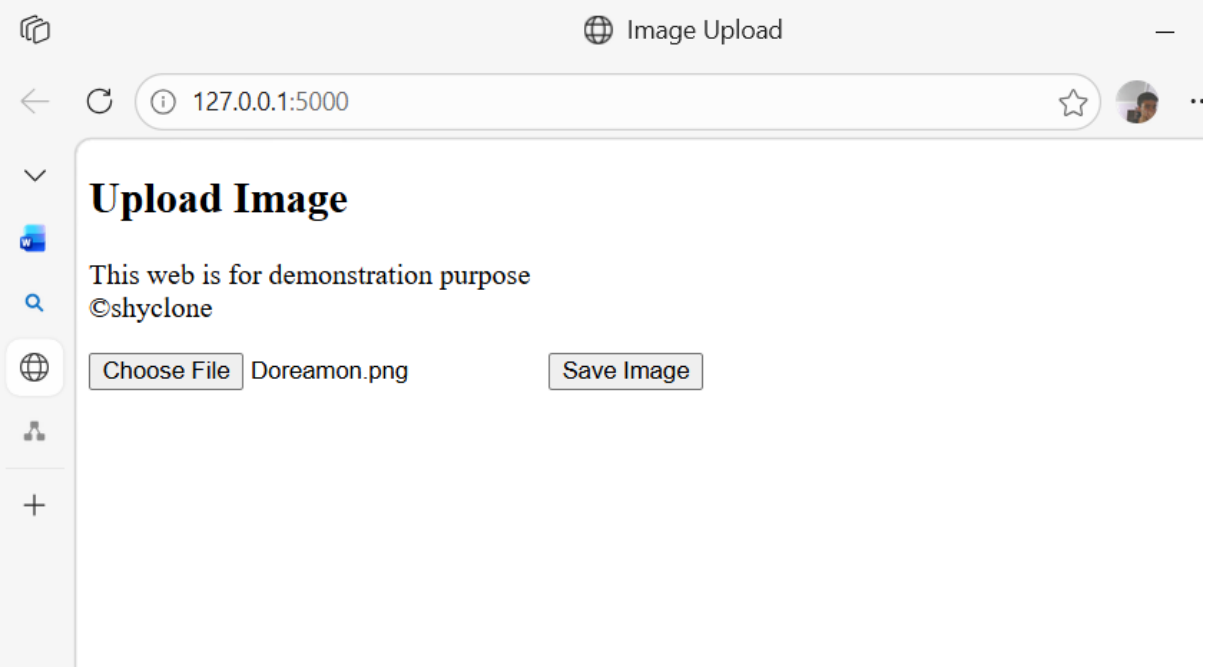

Figure 15: Simple HTTP web

On the client side, the selected image is read as binary data converted into a Base64 string before sending it to server. This ensures safer transmission over text-based HTTP requests.

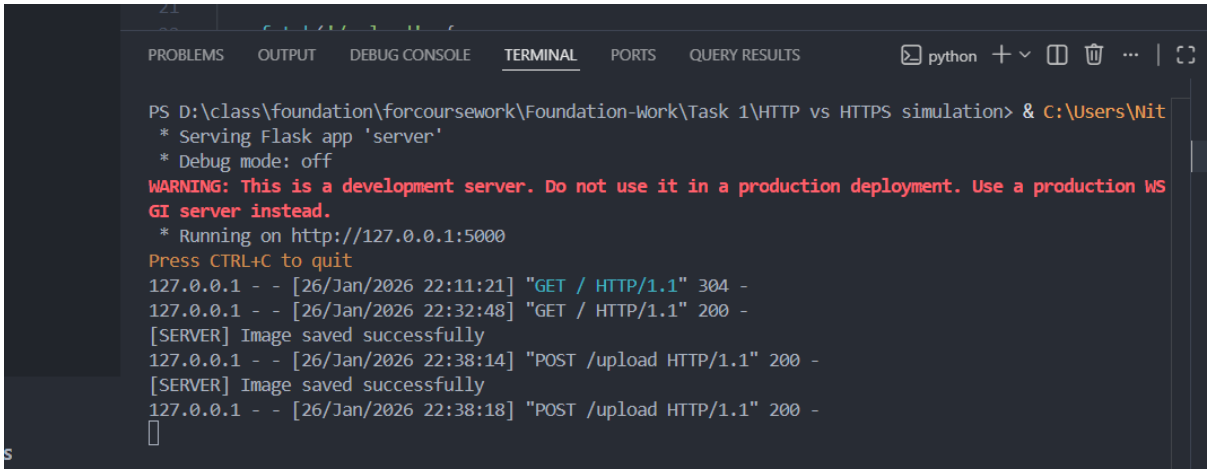

Figure 16: HTTP Server

On the server side, this shows that image is successfully uploaded in server. Since, we transmitted over HTTP, the request payload was sent in plaintext. Attacker monitoring network traffic was able to observe the encoded image data directly.

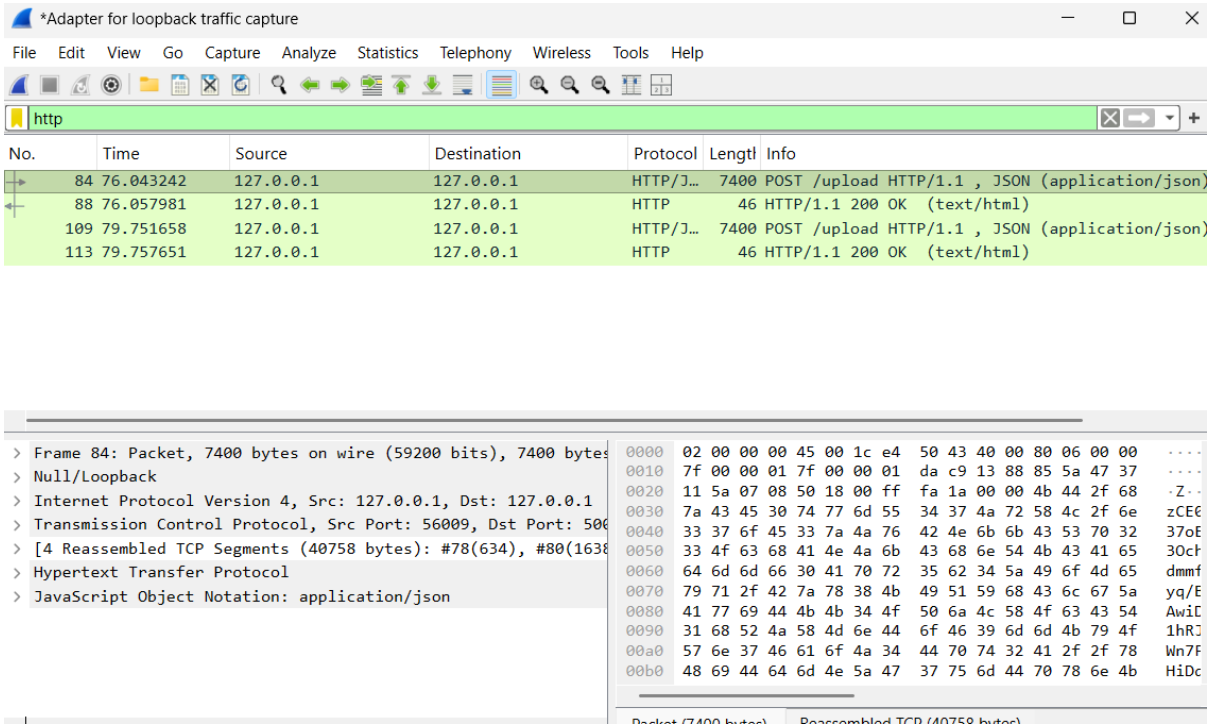

Figure 17: Attacker view when data transmitted.

Attacker can easily view the plain Base64.

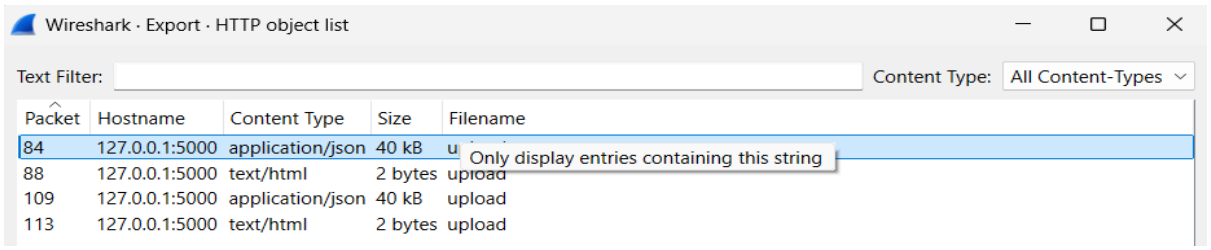

Figure 18: Attacker view when file is uploaded

{"image":"iVBORw0KGgoAAAANSUhEUgAAAUoAAADICAYAAACOJqhiAAAAAXNSR0IArs4c6QAAAARnQU1BAACxjwv8YQUAAAAJcEhZcwAAEnQAABJ0Ad5mH3gAAHUXSURBVHhe7V0FdBRXG32z8RA0QIgHd3d3LVCc4k5xK1Dc3a24e3EvFG+LFfcQHI
KnuBa5/7lvMstmsgkBEvonmXvOd5LdefNmdnbnzvc+FTBgwIABA+FC6NBwYMCAAQMhYRClAQMGDHwCBlEaMGDAwCdgEKWBb4b3b5/j4oUz2P37ZqxbsQRrls7C3MlDMKh7G3Rs2QjtmzVCt7ZN0a5ZQ7Rv3hijBvXH7ElDsGzOKKxaMgOb1qzAzesBwI
f3
+qkNGIhSGERpIMpwS9ZNnDp+DGt+XYDuHZvg+wolkCVDaiRKnhE2KXPCNncFiCzFIPJWg8jzA0TOGhD56kLkqAGRqyZEgfoQWcrDlLMcbPNWhkOWksidMweqVqqARnVqY9Tgbti3dxcZp/VH9qAgUiFQZQGIhVPHv+D8SMHoVuX9ihSJD980mRAPK/0
kvBEnfEQjadCdFwNMfggxFh/iImXIaYGQsy4o8r02x//n3oTYuJViAmXIcZfhBh2GKLRdIg6YyHy14XwSI5kqdIjefrM6PlzF6xcMhfXL57Ei2cP9adlwMBXwSBKA1
+Nh/88wN5t6zCkZ1tkSZsSSpqiEEUaQTSeAvHzBoiRJyF+uQkx9yHE3HBgZt+HmPMAYtY9iFl3IWYGEyP/aqK95nYp99R95gSpc/C9SZchRpyE6LweIn89ZPtmRzyf1ChVOCN+GT0Yf+zcpj9VAwa+CAZRGvhi7NmxAQO7dUT54sWQKIUfRJZyECW7QY
w9C9PUyxBzgkmRpDbjNsS0QAu5
+YUSvP/0WxAzNQL9B2LKFZWQu26A+K4bRPqicEudFsN7dsPuHdvx4b1h1zTw5TCI0sBn49bNG5gzfSISJ/GEyFAa4rs+EH12Qcy+B7HwiUpgJLJQJBeFQiImac4OgrLgMZRp1yA6r4ZIUwKufunw/XflMXH0QFy5eEb/cQwY+CQMojQQYdBpMnrwAOQv
mBf2CZNAtF+l2hHnP1ZFao1WSOybS6C6bJ/3COKXqxDdNkIUagxT/MTIkj0rjv/9J969fav/eAYMhAmDKA1ECH26toW7leER26IWkMhhh9XbYZm7fGGFcL6PxBqmZpddNwFiFYlkSCZJ1o3rI4De3/Hhw8f9B/VgIFQMIjSQLi4fCkAg7r/BJE8H0SD
8RBjzkDMuq8Sj56ULIXapXTaBH0UktZ/pXWSzKU9MwiOXZZAyVISST3c0aZpI+zeuRWAQZgGwoZBlAas4t83b7Bg5khky5gWwj07xMQr6lKWZBOeM4ZESO/2L9chBv4F0WcnRO8dEN03Qww9rI4haUpNNJx5olLmPYLtzNsQrZZAZKoI1yRuqF+jsv4S
GDBghkGUBkLhr717ULd2LTgmcIMo2WFiwvlPa5DU2GZzXX4PovpAiMItYOeXG85eqRHPLz0S+KWDXfJcamB5k4lwHH0kWMP8xk4fTUjU8x9BmXoDouU8CM88mDh6EB4
+uKu/HAYMGERpICSG9OmOuK5uEIWbQvTdCUUumx+EJho96cwLghhyAKJMZ6RIkQItmjXD7OlTsWX9Ghw7uA/nTx3H7KlTUK92DanBieQFIDotU7XU/8y+qTl9HkNMvgybFLlRMEcarF+
5QH9ZDMRyGERpwIy9u7dBJEkJUXeUSiRcQn8qzGfWPZhm34aoNxnCNTVyZPTFuTMn8Ob1a/30Em9ev8LWzWtRunhRiHhJIRqMU4PL9fN+U1EJUxl9DCJvLSRKFA99urXFndtX9KdvIJbCIEoDePv2DebPmABPHx+Ifn9CLHgaIaeLwqX2uPMQhRsgnks
idGzVAlcvh4xTpFdZE0s8fvQIg/r2gqNLPCjUKpm6aOUY31T4eWY9gGi3EiJpGpTIlxn79u4Icd6xGfwO/f39sWXLFhw+fBgvX7UD4mxiNVE+eTJE6xYsQ3jx47FuHHjMH78eKsycuRIrF+/Hs+ePdNPEQL379/HX3/9hUWLFmHChAly3smTJ2PNmjU
4fvw43rx5o98lFB4/fizPiedjeU7a/wsXLs5+ffvw9Olt/a5fhPv37q1b+x/hlNQdovYgiHnUIiPiZAmEGHEcIn15eLomwJIFc/AuOPvFkhzDEgO0D+vWB6LpeJd1Qx7AQ2jK1NEbaSzmef7ns14/9YrmhLsUXPYcYchAiVWn4unsYMZfBePfuHbp37w4
nJyeUKVMGFy5c0A+JsYjVRPnnn38iQ4YMEEJ8UgoUKIDzS8/rp5B48OABfvnlF1SqVAlp06aFg4NDiH1dXV2RI0cOtGzZEtu2bcPbcG48Em2qVKlCHV8T/kh5zj/88IMk74iQb1gIOH8axfLmgnBBNB9GFZEWPtp48whDGJqYrhtxZM+Dvg3
+a59QTYnhCPHv2HCJ1EdVLbo30qNnShkhinHgJYtABiAH7IYafgBhzFo7TbBJm4bPQ+
32NTA+EMvs+FAbTVxmALm0a4enTxxZXLnaCv9tWrVrJ32HOnDlx8uRJ/ZAYiihNlCQaLy8v+cW7uLggefLKVsXPzw8NGzbE9evX9VPgyJEjqFevHhImTCjnURQFSZMmNe/r6ekJZ2dnM9GRSKkdPnxovcLN5s2bJbFybLx48eDr62uey9vbG3HjxjXPx
ffmzp2LF//9Vz/NJ/H3/t3InTMTRIYSEAP2QmFspDWisiazH0CZcAG5s2bCiePHzHPqifBTosHeIR5Ex5Uy9fDjcQKhyID2OxA//wZRuBlEmiJIkK0YkuQqgXg5S8nYTptsFSDqjQ5OnYxEWye1amqwJEsXdzSo/j1u375hPufYCBJl27Zt5W8vb968O
HXqlH5IjEWsJsoNGzZIIuIX37RpU1y9elWSIf9ayuXLl3Hv3j259LDEiRMnUKJECTNxZcqUCYMHD5aaKve7du0ajh07JpfLNWrUgKOjoxyQOHFiTJw4Ea+tOOxo/yHRclybNm1w7tw5OQ/l7NmzWLlyJcqXLw8bGxs5JmvWrHJZ/7lIndIbIn8dmKZeg
jI3KILL7eBMl6nXZd3Ivw/uN8+n38GIClGxWCGIIi1VLVVzHlGD7L8XolBD2MdLgjIFcqJHp3bYsWU99u3+DRtWLsGw/n3QrP4PS38ZBcR3AyDGXwg2HUSivZMPj65bIXwKIn/O7Dh6
+JDFVYXdMIgylsKSKH/66Sf95nDx4sUL+aMxmUxy/yJFimD37t1hand3796V9h1qrhyf3k0aOV4PS6Ls3bu31WU6CbN06dJyj2dnbRd6kk8PGxetwIif33YTjqrerb15BCWTL8F23lBsKs3BjYmk5yLXKcnv88RYkiv7lBSF4WYfEUlJi63G0
+EydUHObNKxsKZ0xAUdF/3KVTwel+
8cA7J3RNDpCoCpf0CKLOCS7Tpz/9LZeEzmAb/BZGyGHJkyoIzp07oTyNWwCDKWApLouzUqZN+c7jYunUrfOglDtYkrZGeHtRKuYTXtEGSs94pY0mUPXr0kIRsDXQUafNQG3716pV+iFVsWL0Enm7xP1bb0chgylWIiQGqkLD0ZEGZGwRl4B8wJfRD57a
t5Hy8eXiOz58/l39pM9WToV60fSjv37/HiiXzETd1LoghR2RcJTVcxzgu6NetDa5fuyw/G8cRlgRLjZxzcD7/c6dQpXRJKEpciNrDYZp5y7pmyQDzSZfUzznpolo0Qz9GL9OZzfMPlNGnINJXQFofN/y5d4f5nPTg+zxnXpN2ZA5jp8hvAcd56OHWfu
sYYFzaOP4N6yHtiW0uXkttetKaOdvOUdEiZJzcV/OYTlndIZBlF9AlPwhcDz3s7W1xdChQyP0oyS2b98Od3d3uW+
2bNlw5kzIcJqIEiU945pdtFatWhEK1di4eglS+nqr2TYaSY44AtFoPESB2hBpCkCkzgeRuwpEzQEQ/XaqxMJxOut8V1bhKZAr044fOybPoXPnztIUQI26VKlS+PHHHzFv3jwEBASEIkhN+BnLli0rZc+ePdixdSM8MhWAGHgINgseQdQfgYmjhmPduvW
oUKECKleuJB9ElnPwAdOzZ09p+uB58IbfsGE9MmXIAMXGHopvZijtFshAcnn+fBD8vBGiYheIrGUgUuSCSFsQonhTiJYzIEYdD02Qmow9A9FqNkTJFlBS54VwjI/48eKiV69e2LVrV6hrz9XDzz//jIIFC6Jdu3bS2WcNR48eRZ06deS1Y6REWKTy+++
/o1y5cnIc/9fj9u3b8hp07dpVfheFCxeW15aOF9qwL168qN/FDJqG6ITkA5wmJhLczJkzUbFiRRQqVAjt27c3OzEjQpS0sfP3mDt3bgwbNkxGccQEGET5BUR3EuCPlvvRWXPw4EH9kDDBm4ihFdyXHmxqppaIKFHSy25vby/HdenS5ZOay8kj++Dr4wl
RouPHAO+f1kBkKgHh9NFBZBYbW4jk2SFazVFJhhk6vbbDPlEy9O3VFd9//72ZqPVC8wJv1o0bN8qbS0
+UkyZNMo/lTXni6EGky1MKov8BiPZLodg4ynMeM2aM1Jpp3uDntZyDoVjadSRZk2hor7U8DyVzSYjR31WyrD0YIplFNIGimkyk8PPX7B+aICm9t0HkqwERx+KzKor5f37/ffv2leejgY46RiVwe7JkyWQ4lzWMGDFCmk44juRiOYcGfq9ceXAMnYL63w
ujPhdJEqUKMRn14TfBSM2+F1Y04BJbHHiXIGbm5u0pfM3Fz9
+fPP+VARmzZolx3L/8Ihy79698n1up9NxxowZVu3w0REGUQYTJckmovjjjz/MWiG1hlu3bumHhAlqHx06dJD70kM+derUEMsuS6KkxmJNU6UGUbVqVfPNs3z58JC1EcL//Dnky5UdokAdKLz5GX9I7YpalXZTpcwNUaQBRInmEDkrQST0UN/PXkGSjUI
NNGtNpPLzQaFCBc375cmTB82aNZNk1bx5c/la20aTxKZNm+TnsyQ5xpZqt10S5f4/9yBFnrIQbRdASV0AzerVkec9evRo6QAjmXAfyzloxqC2yTmomTMCgGOpSTVq1AiuCRNAeGZVSbLjMohEanSD8MkMUb49RJ1hEFV7qSSYIBlEiWahSXLgHxAZi6n
72TlAZCkN8X13iAajIYo2RWJXlZxINFxVaA81nh9f8/vlw2zklCmhvh8SYP369c3XKl26dFYJlU48jXz4ea9c+ZgtRKchf3/aHLly5ZLfBX/LDEVjSJq2LX369PK3pSdLvscoC5IjvzsSK8mfGia/T/7O+B0S/B7DIkpqpnw4cluSJEnk90XtNKbAIMp
gouQSSbOb6YXkZklmXOZosZL8YX7O8oI3zKhRo8xE0a9fvxBLN0uipCbBZZt2Hnfu3JE/WmoffNJzIIMGDSRxhoWg+/dRqmB+iFQl1eXlnPsQ45lNE3yTUnPk8pPL7F+uqQQx5hREh6UQuSpDFPwBYvQJSRq2rsmQO0d2uR8/P28aev55/vxc/Mubh0s
+assc R+Liza4nSmqKJBJqK9u3bYJnuqwy/CdtxgwIvHlZjqNGyXksycaSKllk1IiAY3i9GG3A7b8uW4T48ROrJgMSI8fTwrRem5IMhx3TiXSHptCvk8tukxrVfN0iANRoSPEiKMftzN0yTU1Uvh5y7kZQmZpp2a8LN/jttq1a8vkBksmoGannb+vB6
zZ88OMYbgUlv7PVBz1WzR/F1w2c73eY348GVco7adRMxoCC6dte/iu+++k9+FJfh7ozbJ74JjUqdOjWXLlpltjDdu3DCfu37pffr0afk+P0vlypXl+yRJfld6cORh0GUwUTJeMpixYqFEj4aXuj1qaBWqBGVLRFfe6Tk8vIsPbXfrjcRmcRj6
+dS758+eTNR5KiFkAnDlPKwsPPndpDJEwJpc/vwQUogpfcmsaYvjDE4H2htSkK7ZeD/lSDwWsOQfLkPjk0iftVqVJF3kSW5KUJbV3asjhBggT49ddfpSYTFlEuWbwQTjJ0yoTJk6fI8/4couRDhw+MwMBAuY14
+/Zf1K1ZFWKCP0eVfsWftnUwh36z2lNqHG7p1b3v1YutA2TOentl8DGJOFc4sSR44YMGWI2gdDEOrsi38+SJUuo4OxVq1bJa0kNfkVEOI5kp3/o0s7H75tLWYaGaeDDWvsuuPS2FuNL8MFRsmRJOY7H4hzaNSIsf2

Figure 19: Image in Base64

This can be easily decoded by Base64 decoder.

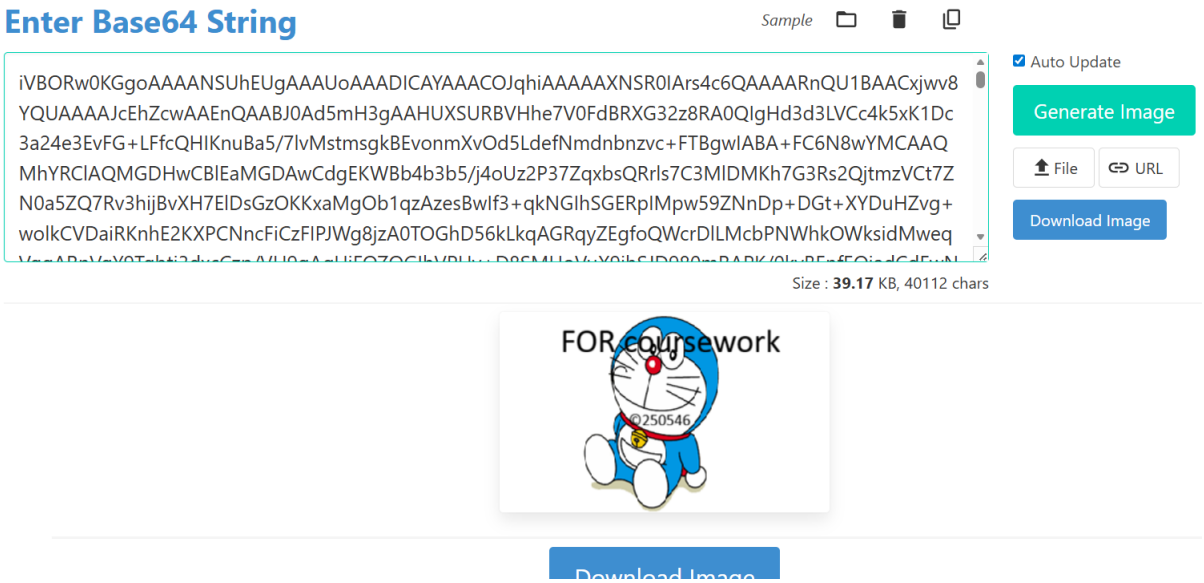

Figure 20: Image Transferred.

This shows that Base64 encoding improves data compatibility but does not provide confidentiality.

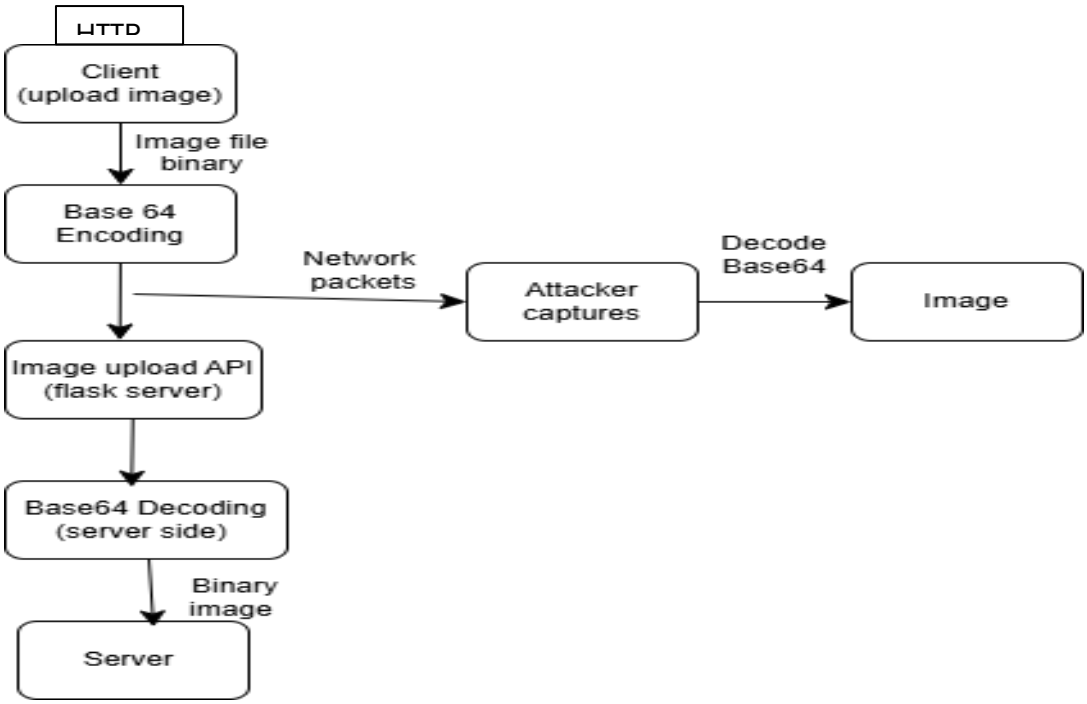

Figure 21: Data flow diagram of HTTP

Image transmission through HTTPS

Https ensures safe transmission of data through the internet. Below is the demonstration of how https securely transfer data.

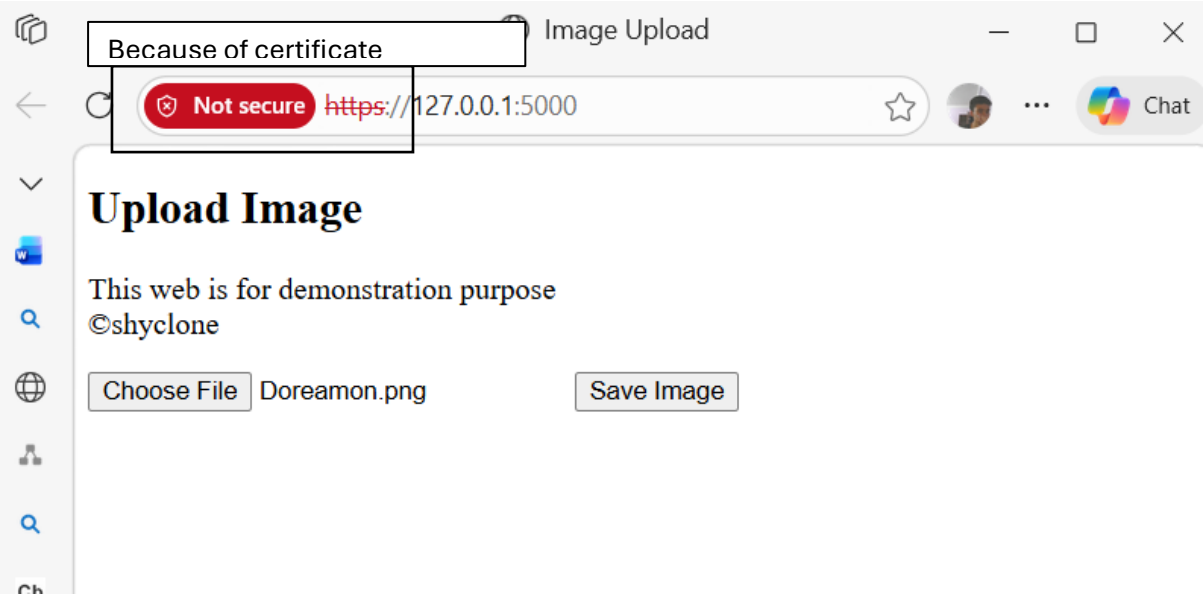

Figure 22: Simple HTTPS web

On the client side, the selected image is read as binary data converted into a Base64 string before sending it to server. This ensures safer transmission over text-based HTTPs requests.

Figure 23: HTTPs server

On server side, this shows the successful transmission of image from the web. When attacker captures the data, they won't be able to see what is transferred.

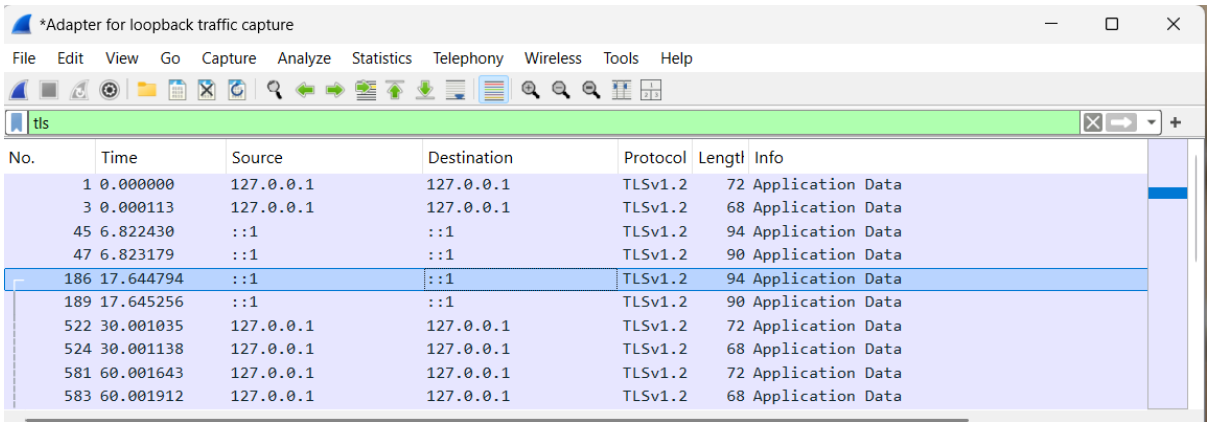

Figure 24: Attacker view when they capture HTTPs packets

HTTPS add a layer of security to data being transferred over the internet. It provides safety features Encryption, Integrity and Authentication. It is possible by SSL (Secure Socket Layer) certificates and the modern TLS (Transport Layer Security). (Freeman, 2024) Due to these features, even if attacker captures the data, they cannot decrypt the data.

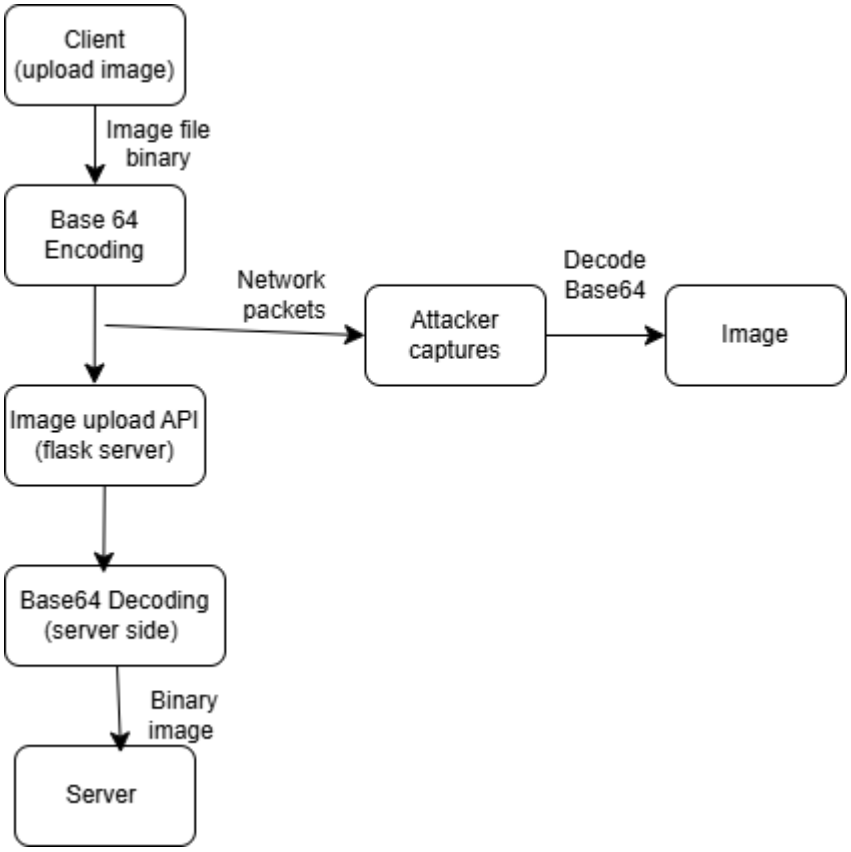

Figure 25: Data flow diagram of HTTPS

Reference

Bagci, T. (2025, September 8). What is ASCII? | Definition, Types, Properties & Character Table. SYSNETTECH Solutions. https://www.sysnettechsolutions.com/en/what-is-ascii/

Josefsson, S. (2003) RFC 3548: The base16, Base32, and Base64 data encodings, IETF Datatracker. Available at: https://datatracker.ietf.org/doc/html/rfc3548

OnlineTinker. (2025, November 23). Demystifying Base64: How Encoding Works and When to Use It - onlinetinker blog. OnlineTinker. https://www.onlinetinker.com/blog/base64-encoding-guide

Hexdecoder (2024) Hex Decoder Beginner's Guide - Complete Tutorial for Hexadecimal Encoding & Decoding. Available at: https://hexdecoder.com/guides/beginners/

Berners-Lee, T., Masinter, L., & McCahill, M. (1994). Uniform Resource Locators (URL). https://doi.org/10.17487/rfc1738

Ibrahim, M. (2024, August 2). *What is JWT? Understand JSON Web Tokens | SuperTokens*. https://supertokens.com/blog/what-is-jwt

Freeman, R. (2024, October 11). *Understanding the key difference between HTTP and HTTPS*. Hosted.com Blog. https://www.hosted.com/blog/difference-between-http-and-https/#what-is-https

Saeed, H. (2023) *The search for efficient algorithms: P vs. Np and heuristics | by Husnain Saeed | Medium*, *The Search for Efficient Algorithms: P vs. NP and Heuristics*. Available at: https://husnainsaeed000.medium.com/the-search-for-efficient-algorithms-p-vs-np-and-heuristics-8c828772b0e3